

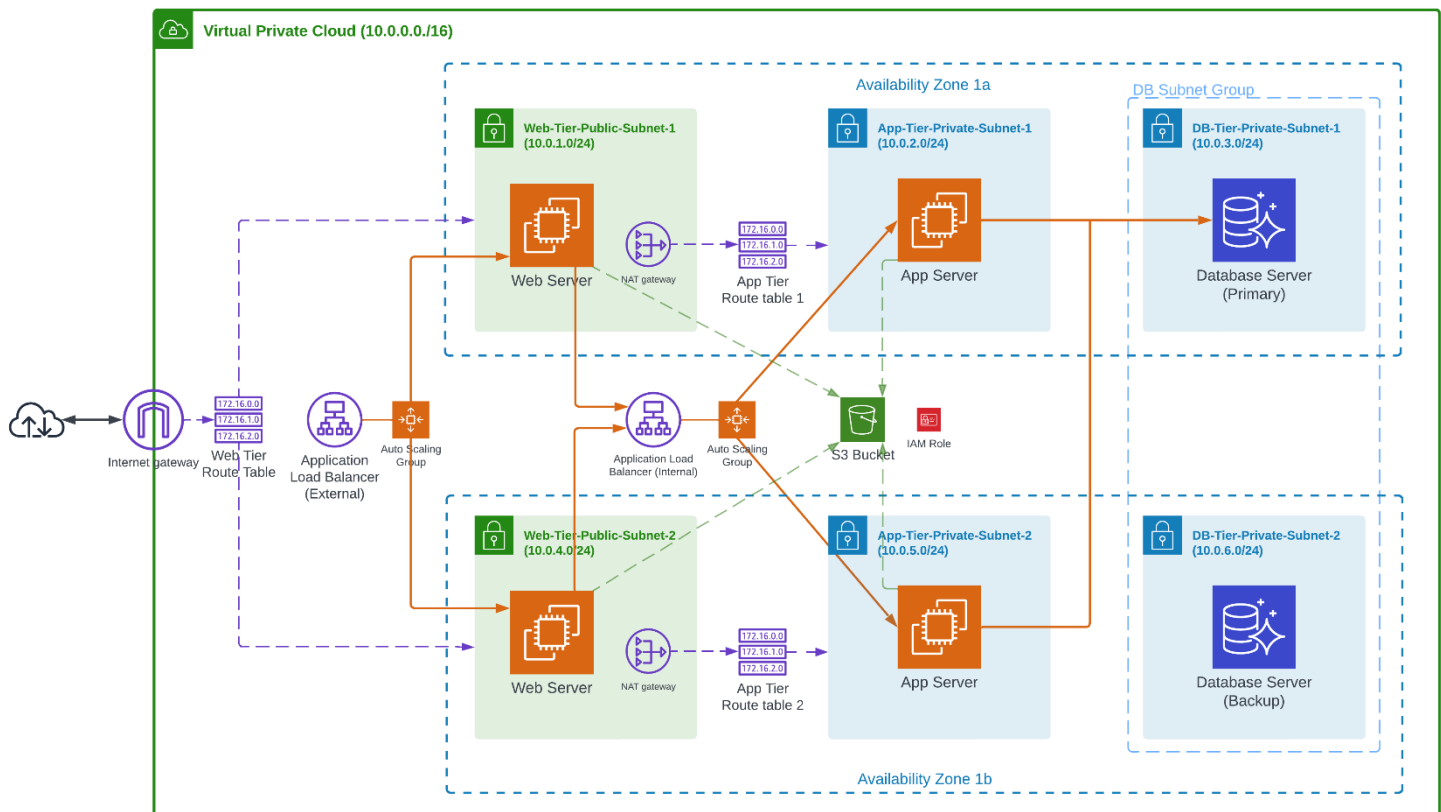


DEPLOY A THREE TIER WEB APPLICATION ON AWS

Shehan Arampola

<https://github.com/shehan404>

<https://www.linkedin.com/in/shehan-arampola>



The three-tier web application architecture is a robust and scalable design pattern that separates an application into three distinct layers: the presentation/web tier, the application tier, and the database tier. This architecture enhances security, manageability, and flexibility by isolating each layer's responsibilities.

In this documentation, let's see how to deploy a three-tier web application on AWS

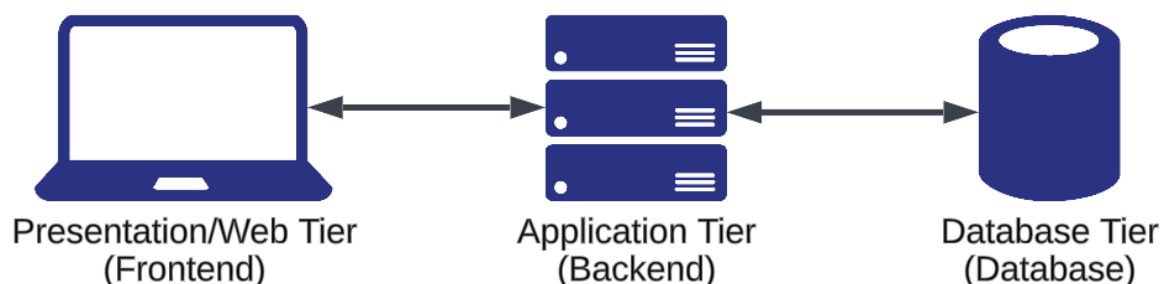


Introduction

A three-tier architecture is a common architectural pattern used in software development and deployment, particularly in cloud environments like AWS. As I mentioned before, it separates an application into three main layers or tiers: the presentation/Web tier, the application tier, and the database tier.

1. **Presentation/Web Tier (Frontend)** - This is the user interface layer where users interact with the application. It's responsible for presenting information to the user and collecting user inputs.
2. **Application Tier (Backend)** - This is the logic layer where the actual application logic, processing, and control flow take place. It processes user inputs, performs calculations, and makes decisions.
3. **Database Tier (Database)** - This layer is responsible for data storage and management. It stores, retrieves, and manages data that the application tier needs.

Users interact with the application via the presentation tier. These requests are sent to the application tier, where the logic processes the request. The application tier interacts with the database tier to retrieve or store data. The processed data or output is sent back to the presentation tier, which displays the results to the user.



Project Overview

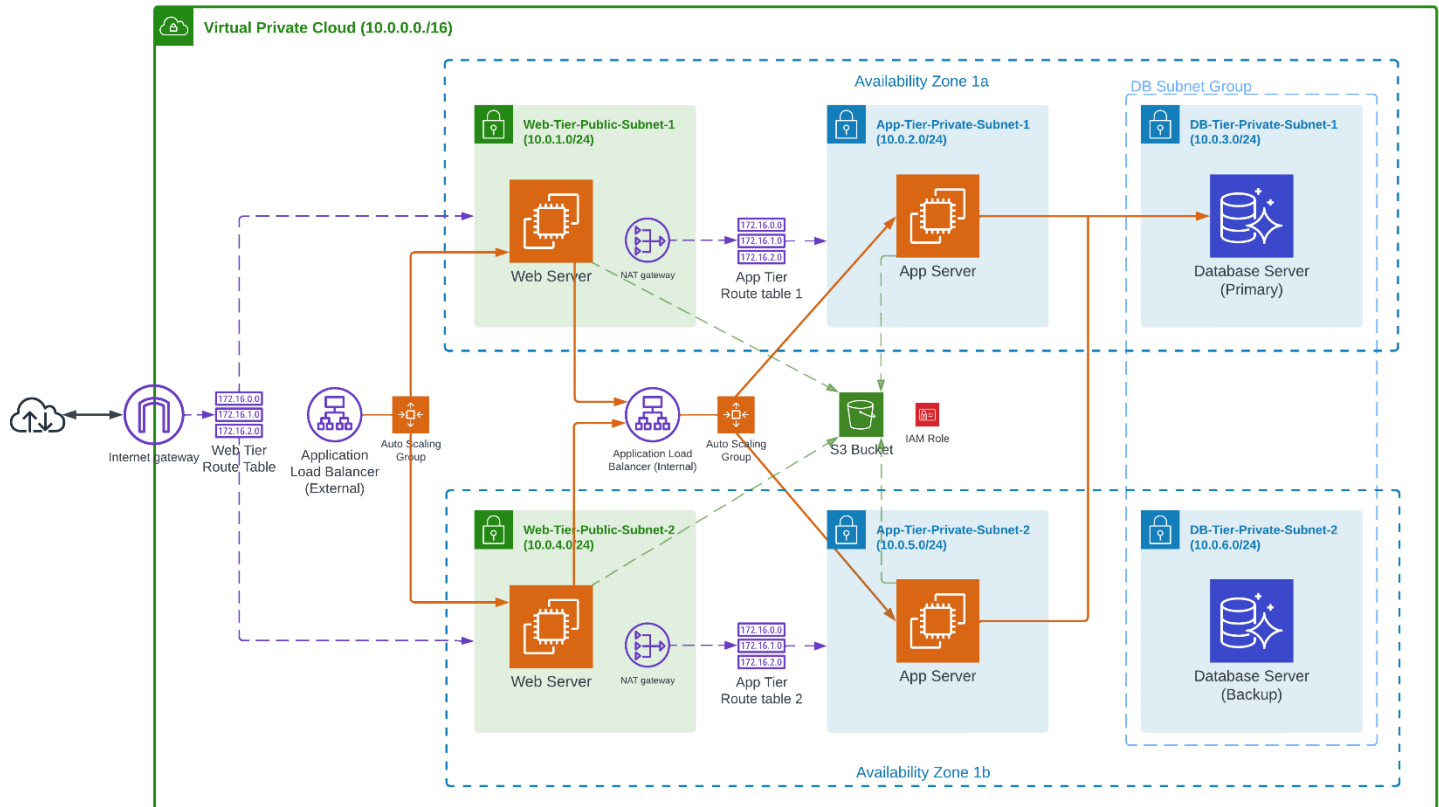
In this project we are implementing **Highly Available, Highly Secured, Highly Scalable** and **Fault tolerant** three tier application that AWS

Throughout this project, we have to interact with the following components on AWS.

1. Amazon Elastic Compute Cloud (EC2)
2. Amazon Simple Storage Service (S3)
3. Virtual Private Cloud (VPC)
4. Auto Scaling Group (ASG)
5. Application Load Balancer (ALB)
6. Identity and Access Management (IAM)
7. Amazon Relational Database Service (RDS)
8. Amazon Machine Image (AMI)
9. Launch Templates
10. Target Groups
11. Internet Gateway (IGW)
12. NAT Gateway
13. Route Table
14. Security Groups



The following diagram shows the overview of the project. It shows connection between each component and how they are distributed in two availability zones. Let's have a look at architecture diagram



As you can see, we use public subnets for web tier because it should be open to the internet to let users interact with the application. App tier and Database tier should be secured. Therefore, private subnets are assigned for them. Resources are distributed in two availability zones, and all are included in our custom VPC. Observe how CIDR blocks are assigned for each subnet.

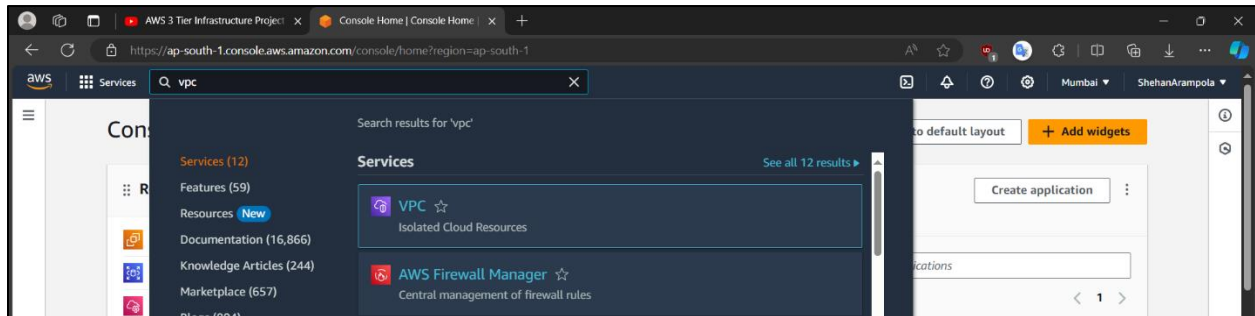
An S3 Bucket has been used to store the source codes of the application. I will make the process of server configuration easier.

Source Codes: https://github.com/shehan404/Three_Tier_app_on_aws

Let's get started with the implementation 🤖🔧🚀

Project Implementation

Go to **AWS Console** and **choose a region** you like (I choose Mumbai region) and create the following services as shown in figures. For go to each services use search bar and search for them.



Note: When inserting names when creating components, you can always use names as you like

1. Create VPC and Subnets

Create VPC

We have to create a VPC. Search for VPC and go to **Your VPCs > Create VPC**. Setup it as follows. Use **10.0.0.0/16** as CIDR block.

A screenshot of the AWS 'Create VPC' console page. The page title is 'Create VPC' with an 'Info' link. Below the title, there is a description: 'A VPC is an isolated portion of the AWS Cloud populated by AWS objects, such as Amazon EC2 instances.' The 'VPC settings' section contains the following options: 'Resources to create' (radio buttons for 'VPC only' and 'VPC and more', with 'VPC only' selected), 'Name tag - optional' (text input field with 'Three-tier-project-VPC'), 'IPv4 CIDR block' (radio buttons for 'IPv4 CIDR manual input' and 'IPAM-allocated IPv4 CIDR block', with 'IPv4 CIDR manual input' selected), 'IPv4 CIDR' (text input field with '10.0.0.0/16'), 'IPv6 CIDR block' (radio buttons for 'No IPv6 CIDR block', 'IPAM-allocated IPv6 CIDR block', 'Amazon-provided IPv6 CIDR block', and 'IPv6 CIDR owned by me', with 'No IPv6 CIDR block' selected), and 'Tenancy' (dropdown menu with 'Default' selected). The 'Tags' section has a table with two columns: 'Key' and 'Value - optional'. There is one tag with 'Name' as the key and 'Three-tier-project-VPC' as the value. At the bottom, there are 'Cancel' and 'Create VPC' buttons.

Create Subnets

Next, we Have to create six subnets in this VPC. (Refer the diagram)

In Availability Zone 1a

1. Web-Tier-Subnet-1-Public
2. App-Tier-Subnet-1-Private
3. DB-Tier-Subnet-1-Private

In Availability Zone 1b

4. Web-Tier-Subnet-2-Public
5. App-Tier-Subnet-2-Private
6. DB-Tier-Subnet-2-Private

Go to **VPC dashboard > Subnets > Create subnets**. And setup them as follows.
And Click create subnets

VPC > Subnets > Create subnet

Create subnet [Info](#)

VPC

VPC ID
Create subnets in this VPC.
vpc-0c733d1887af88883 (Three-tier-project-VPC)

Associated VPC CIDRs

IPv4 CIDRs
10.0.0.0/16

Subnet 1 of 6

Subnet name
Create a tag with a key of 'Name' and a value that you specify.
Web-Tier-Subnet-1-Public
The name can be up to 256 characters long.

Availability Zone [Info](#)
Choose the zone in which your subnet will reside, or let Amazon choose one for you.
Asia Pacific (Mumbai) / ap-south-1a

IPv4 VPC CIDR block [Info](#)
Choose the VPC's IPv4 CIDR block for the subnet. The subnet's IPv4 CIDR must lie within this block.
10.0.0.0/16

IPv4 subnet CIDR block
10.0.1.0/24 256 IPs

Tags - optional

Key	Value - optional	
Q Name	Q Web-Tier-Subnet-1-Public	Remove

Add new tag
You can add 49 more tags.
Remove

Subnet 2 of 6

Subnet name
Create a tag with a key of 'Name' and a value that you specify.
App-Tier-Subnet-1-Private
The name can be up to 256 characters long.

Availability Zone [Info](#)
Choose the zone in which your subnet will reside, or let Amazon choose one for you.
Asia Pacific (Mumbai) / ap-south-1a

IPv4 VPC CIDR block [Info](#)
Choose the VPC's IPv4 CIDR block for the subnet. The subnet's IPv4 CIDR must lie within this block.
10.0.0.0/16

IPv4 subnet CIDR block
10.0.2.0/24 256 IPs

Tags - optional

Key	Value - optional	
Q Name	Q App-Tier-Subnet-1-Private	Remove

Add new tag
You can add 49 more tags.
Remove

Subnet 3 of 6

Subnet name
Create a tag with a key of 'Name' and a value that you specify.

The name can be up to 256 characters long.

Availability Zone [Info](#)
Choose the zone in which your subnet will reside, or let Amazon choose one for you.

IPv4 VPC CIDR block [Info](#)
Choose the VPC's IPv4 CIDR block for the subnet. The subnet's IPv4 CIDR must lie within this block.

IPv4 subnet CIDR block

 256 IPs

< > ^ v

▼ **Tags - optional**

Key	Value - optional	
Name	DB-Tier-Subnet-1-Private	Remove

You can add 49 more tags.

Subnet 4 of 6

Subnet name
Create a tag with a key of 'Name' and a value that you specify.

The name can be up to 256 characters long.

Availability Zone [Info](#)
Choose the zone in which your subnet will reside, or let Amazon choose one for you.

IPv4 VPC CIDR block [Info](#)
Choose the VPC's IPv4 CIDR block for the subnet. The subnet's IPv4 CIDR must lie within this block.

IPv4 subnet CIDR block

 256 IPs

< > ^ v

▼ **Tags - optional**

Key	Value - optional	
Name	Web-Tier-Subnet-2-Public	Remove

You can add 49 more tags.

Subnet 5 of 6

Subnet name
Create a tag with a key of 'Name' and a value that you specify.

The name can be up to 256 characters long.

Availability Zone [Info](#)
Choose the zone in which your subnet will reside, or let Amazon choose one for you.

IPv4 VPC CIDR block [Info](#)
Choose the VPC's IPv4 CIDR block for the subnet. The subnet's IPv4 CIDR must lie within this block.

IPv4 subnet CIDR block

 256 IPs

< > ^ v

▼ **Tags - optional**

Key	Value - optional	
Name	App-Tier-Subnet-2-Private	Remove

You can add 49 more tags.

Subnet 6 of 6

Subnet name
Create a tag with a key of 'Name' and a value that you specify.

The name can be up to 256 characters long.

Availability Zone [Info](#)
Choose the zone in which your subnet will reside, or let Amazon choose one for you.

IPv4 VPC CIDR block [Info](#)
Choose the VPC's IPv4 CIDR block for the subnet. The subnet's IPv4 CIDR must lie within this block.

IPv4 subnet CIDR block

 256 IPs

< > ^ v

▼ **Tags - optional**

Key	Value - optional	
Name	DB-Tier-Subnet-2-Private	Remove

You can add 49 more tags.

They will create as follows

Subnets (6) [Info](#)

Last updated less than a minute ago

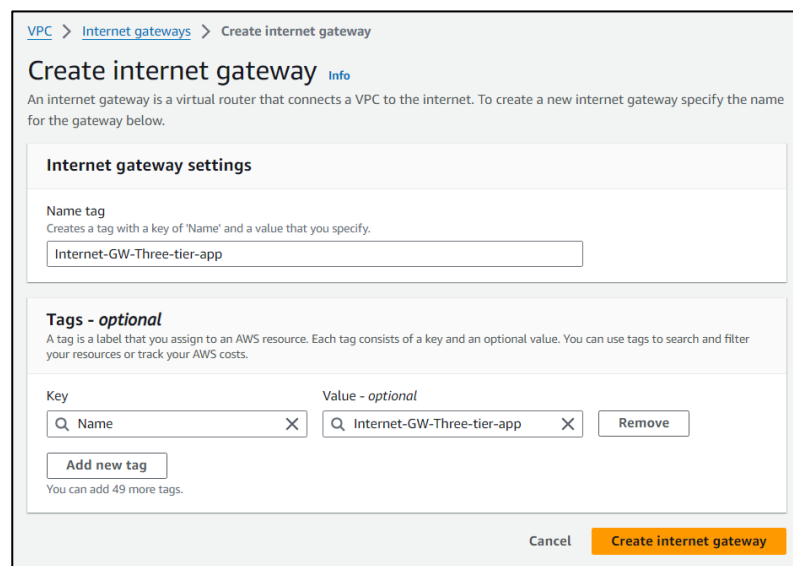
☐	Name	Subnet ID	State	VPC	IPv4 CIDR	IPv6 CIDR
☐	App-Tier-Subnet-2-Private	subnet-0925e6911290fbad0	Available	vpc-0c733d1887af88883 Thre...	10.0.5.0/24	-
☐	Web-Tier-Subnet-1-Public	subnet-01b03b579d3a79019	Available	vpc-0c733d1887af88883 Thre...	10.0.1.0/24	-
☐	DB-Tier-Subnet-1-Private	subnet-020b8dc5fafd7f225	Available	vpc-0c733d1887af88883 Thre...	10.0.3.0/24	-
☐	DB-Tier-Subnet-2-Private	subnet-0768fb507f77347fe	Available	vpc-0c733d1887af88883 Thre...	10.0.6.0/24	-
☐	Web-Tier-Subnet-2-Public	subnet-0bc57027ba7344786	Available	vpc-0c733d1887af88883 Thre...	10.0.4.0/24	-
☐	App-Tier-Subnet-1-Private	subnet-0631d4ffa45df8f68	Available	vpc-0c733d1887af88883 Thre...	10.0.2.0/24	-

Now select **Web-Tier-Subnet-1-Public**. Go to **Action > Edit subnet settings**. Under Auto-assign IP settings, select **Enable auto-assign public IPv4 address** and save. Do the same for **Web-Tier-Subnet-2-Public** as well.

2. Create Internet Gateway, NAT Gateway and Rout tables.

Create Internet Gateway

Go to VPC Dashboard **Internet gateways > Create internet Gateway**. Enter a name click create.



The screenshot shows the AWS Management Console page for creating an internet gateway. The breadcrumb trail at the top is 'VPC > Internet gateways > Create internet gateway'. The main heading is 'Create internet gateway' with an 'Info' link. Below this is a descriptive paragraph: 'An internet gateway is a virtual router that connects a VPC to the internet. To create a new internet gateway specify the name for the gateway below.' The form is divided into two sections. The first section, 'Internet gateway settings', contains a 'Name tag' label and a text input field with the value 'Internet-GW-Three-tier-app'. The second section, 'Tags - optional', includes a description of tags and a table for adding them. The table has columns for 'Key' and 'Value - optional'. One tag is already added with the key 'Name' and the value 'Internet-GW-Three-tier-app'. There are buttons for 'Add new tag', 'Remove', and 'Cancel'. At the bottom right is an orange 'Create internet gateway' button.

VPC > Internet gateways > Create internet gateway

Create internet gateway [Info](#)

An internet gateway is a virtual router that connects a VPC to the internet. To create a new internet gateway specify the name for the gateway below.

Internet gateway settings

Name tag
Creates a tag with a key of 'Name' and a value that you specify.

Tags - optional

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key	Value - optional	
Name	Internet-GW-Three-tier-app	Remove

You can add 49 more tags.

Then Go to internet gateways list and select our internet gateway and go to **Actions > Attach to VPC**. Then select our VPC.

Create NAT Gateways

Go to VPC Dashboard **NAT gateways** > **Create internet Gateway**. Create it as follows.

NAT gateway settings

Name - optional
Create a tag with a key of 'Name' and a value that you specify.

The name can be up to 256 characters long.

Subnet
Select a subnet in which to create the NAT gateway.

subnet-01b03b579d3a79019 (Web-Tier-Subnet-1-Public) ▼

Connectivity type
Select a connectivity type for the NAT gateway.

☒ Public
☐ Private

Elastic IP allocation ID [Info](#)
Assign an Elastic IP address to the NAT gateway.

eipalloc-05ab6abf8e395e1f0 ▼ Allocate Elastic IP

► **Additional settings** [Info](#)

Tags
A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key	Value - optional	
Name	NAT-GW-1	X

You can add 49 more tags.

NAT gateway settings

Name - optional
Create a tag with a key of 'Name' and a value that you specify.

The name can be up to 256 characters long.

Subnet
Select a subnet in which to create the NAT gateway.

subnet-0bc57027ba7344786 (Web-Tier-Subnet-2-Public) ▼

Connectivity type
Select a connectivity type for the NAT gateway.

☒ Public
☐ Private

Elastic IP allocation ID [Info](#)
Assign an Elastic IP address to the NAT gateway.

eipalloc-03645ed7f2efe9028 ▼ Allocate Elastic IP

► **Additional settings** [Info](#)

Tags
A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key	Value - optional	
Name	NAT-GW-2	X

You can add 49 more tags.

Create Route Tables

Now Let's create Route tables. According to our diagram we have to create three route tables.

1. App-Tier-RT-1
2. App-Tier-RT-2
3. Web-Tier-RT

Now got to VPC dashboard **Route tables** > **Create route table**. Create Rout tables as follows.

Route table settings

Name - optional
Create a tag with a key of 'Name' and a value that you specify.

VPC
The VPC to use for this route table.

vpc-0c733d1887af88883 (Three-tier-project-VPC) ▼

Tags
A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key	Value - optional	
Name	App-Tier-RT-1	X

You can add 49 more tags.

Route table settings

Name - optional
Create a tag with a key of 'Name' and a value that you specify.

VPC
The VPC to use for this route table.

vpc-0c733d1887af88883 (Three-tier-project-VPC) ▼

Tags
A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key	Value - optional	
Name	App-Tier-RT-2	X

You can add 49 more tags.

Route table settings

Name - optional

Create a tag with a key of 'Name' and a value that you specify.

Web-Tier-RT

VPC

The VPC to use for this route table.

vpc-0c733d1887af88883 (Three-tier-project-VPC)

Tags

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key

Q Name

X

Value - optional

Q Web-Tier-RT

X

Remove

Add new tag

You can add 49 more tags.

Now you will see all the route tables in the menu.

Route tables (5) Info

Last updated 2 minutes ago

Actions

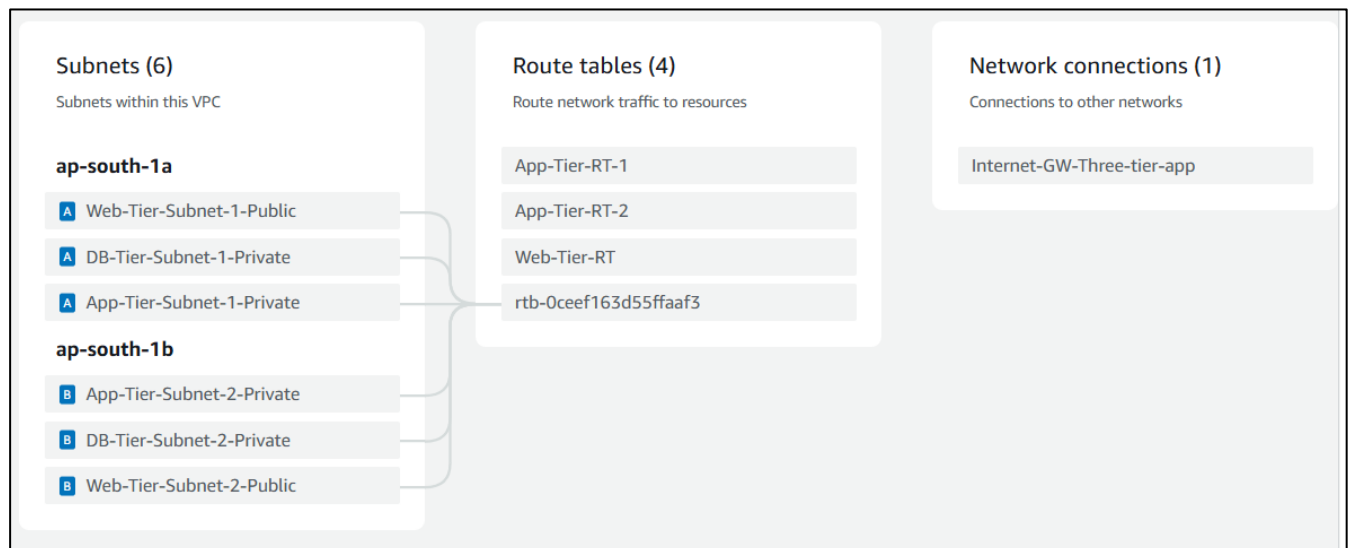
Create route table

Find resources by attribute or tag

< 1 >

☐	Name	Route table ID	Explicit subnet associ...	Edge associations	Main	VPC	Owner ...
☐	Web-Tier-RT	rtb-019d58bbaf8f9f1e1	-	-	No	vpc-0c733d1887af88883 Thre...	6546544...
☐	App-Tier-RT-2	rtb-05c2bf99a1359fdcb	-	-	No	vpc-0c733d1887af88883 Thre...	6546544...
☐	App-Tier-RT-1	rtb-0d901119d09b7d358	-	-	No	vpc-0c733d1887af88883 Thre...	6546544...

If you go to VPC dashboard > Your VPCs. And select Three-tier-project-VPC. Then if you go to Resource map you can see Route tables are created but there are no connections.



Edit Routes

1. Now Select Web-Tier-RT Go to **Actions > Edit routes**. Add route as follows. Destination 0.0.0.0/0, target internet gateway > select our igw. Then Save changes.

Destination	Target	Status	Propagated
10.0.0.0/16	local	Active	No
0.0.0.0/0	Internet Gateway	-	No

Add route

Again, select the Web-Tier-RT and go to **Actions > Edit subnet associations**. Select Web-Tier-Subnet-1-Public and Web-Tier-Subnet-2-Public and Save associations.

2. Now Select App-Tier-RT-1 Go to **Actions > Edit routes**. Add route as follows. Destination 0.0.0.0/0, target NAT gateway > select our NAT-GW-1. Then Save changes.

Destination	Target	Status	Propagated
10.0.0.0/16	local	Active	No
0.0.0.0/0	NAT Gateway	Active	No

Add route

Again, select the App-Tier-RT-1 and go to **Actions > Edit subnet associations**. Select App-Tier-Subnet-1-Private and Save associations.

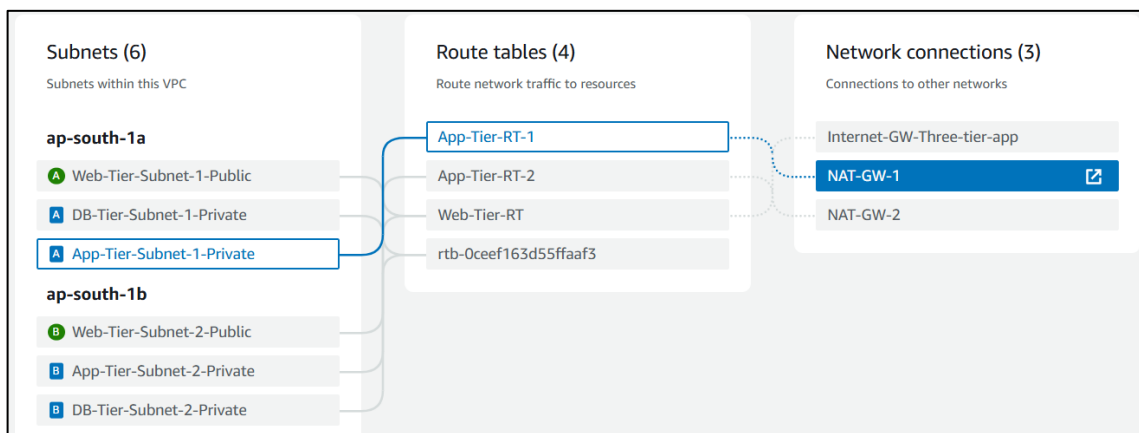
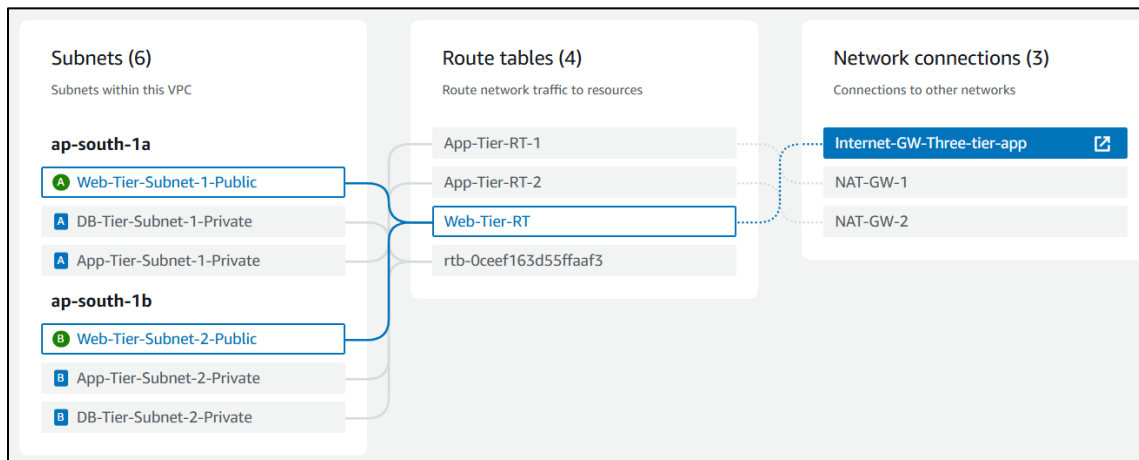
3. Now Select App-Tier-RT-2 Go to **Actions > Edit routes**. Add route as follows. Destination 0.0.0.0/0, target NAT gateway > select our NAT-GW-2. Then Save changes.

Destination	Target	Status	Propagated
10.0.0.0/16	local	Active	No
0.0.0.0/0	NAT Gateway	Active	No
	nat-0c37dd01651f1e648		

Add route

Again, select the App-Tier-RT-2 and go to **Actions > Edit subnet associations**. Select App-Tier-Subnet-2-Private and Save associations.

Now if you check the Resource map you can see that connections have been made. Check if there are any mistakes according to our architecture diagram by hovering over each element.



Good Job 🌟😊. Now Let's Create Security Groups.

3. Create Security Groups

We have to create the following Security Groups.

Security Group	Type (Port Range)	Source
External-Load-Balancer-SG	HTTP (80)	0.0.0.0/0
Web-Tier-SG	HTTP (80)	External-Load-Balancer-SG
Internal-Load-Balancer-SG	HTTP (80)	Web-Tier-SG
App-Tier-SG	Custom TCP (4000)	Internal-Load-Balancer-SG
DB-Tier-SG	MYSQL (3306)	App-Tier-SG

Go to VPC dashboard **Security Groups > Create security group**. Create **External-Load-Balancer-SG** and add inbound rule as follows by referring the above table. Then Click Create security group.

Basic details

Security group name [Info](#)

Name cannot be edited after creation.

Description [Info](#)

VPC [Info](#)

Inbound rules [Info](#)

Type Info	Protocol Info	Port range Info	Source Info	Description - optional Info
HTTP v	TCP	80	Anywh... Q 0.0.0.0/0 X	

Tags - optional

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key	Value - optional	
Q Name X	Q External-Load-Balancer-SG X	Remove

You can add up to 49 more tags

Create other Security groups as well as shown in the table. The process is same. After creating all Security groups you can see them like this.

Security Groups (7) Info					
Find resources by attribute or tag		Refresh	Actions	Export security groups to CSV	Create security group
< 1 > Settings					
☐	Name	Security group ID	Security group name	VPC ID	Description
☐	Web-Tier-SG	sg-0b1f73545c5dd8d03	Web-Tier-SG	vpc-0c733d1887af88883	Web-Tier-SG
☐	Internal-Load-Balancer-SG	sg-0065f4ac04226f8c8	Internal-Load-Balancer-SG	vpc-0c733d1887af88883	Internal-Load-Balancer-SG
☐	External-Load-Balancer-SG	sg-037f8aa1874586aad	External-Load-Balancer-SG	vpc-0c733d1887af88883	External-Load-Balancer-SG
☐	DB-Tier-SG	sg-02b96a2ea03c8b652	DB-Tier-SG	vpc-0c733d1887af88883	DB-Tier-SG
☐	App-Tier-SG	sg-06187cc79e5667ce9	App-Tier-SG	vpc-0c733d1887af88883	App-Tier-SG

Well Done 😊. Now Let's Move on to setup the database servers.

4. Create Database Subnet Group and RDS

First, we need to create a subnet group. But why? 😕

Because, when we deploy an RDS instance in a Multi-AZ configuration, AWS automatically creates a primary database instance in one Availability Zone (AZ) and a standby replica in another AZ within the same region. This setup requires subnet groups because each subnet in the group must belong to a different AZ, ensuring that your database instances are spread across multiple AZs for redundancy and failover protection.

Cool 😊

Now search for RDS in the search bar. In the Amazon RDS go to **Subnet groups** > **Create DB subnet group**. Setup it as follows.

Subnet group details

Name
You won't be able to modify the name after your subnet group has been created.

Must contain from 1 to 255 characters. Alphanumeric characters, spaces, hyphens, underscores, and periods are allowed.

Description

VPC
Choose a VPC identifier that corresponds to the subnets you want to use for your DB subnet group. You won't be able to choose a different VPC identifier after your subnet group has been created.

Choose Availability Zones that show in the diagram (1a, 1b).
Choose Subnet which are assigned to the DB (10.0.3.0/24, 10.0.6.0/24).

Add subnets

Availability Zones
Choose the Availability Zones that include the subnets you want to add.

Choose an availability zone ▼

ap-south-1a ✕ ap-south-1b ✕

Subnets
Choose the subnets that you want to add. The list includes the subnets in the selected Availability Zones.

Select subnets ▼

subnet-020b8dc5fafd7f225 (10.0.3.0/24) ✕

subnet-0768fb507f77347fe (10.0.6.0/24) ✕

For Multi-AZ DB clusters, you must select 3 subnets in 3 different Availability Zones.

Subnets selected (2)

Availability zone	Subnet ID	CIDR block
ap-south-1a	subnet-020b8dc5fafd7f225	10.0.3.0/24
ap-south-1b	subnet-0768fb507f77347fe	10.0.6.0/24

Now go to Amazon RDS **Databases > Create database**. Setup it as follows (In our application we have use MySQL there for we use Aurora (MySQL Compatible) option. We can use MySQL as well).

Choose a database creation method [Info](#)

☒ **Standard create**
You set all of the configuration options, including ones for availability, security, backups, and maintenance.

☐ **Easy create**
Use recommended best-practice configurations. Some configuration options can be changed after the database is created.

Engine options

Engine type [Info](#)

☒ Aurora (MySQL Compatible)



☐ Aurora (PostgreSQL Compatible)



☐ MySQL



☐ MariaDB



☐ PostgreSQL



☐ Oracle

ORACLE

☐ Microsoft SQL Server



☐ IBM Db2

IBM Db2

Engine Version

Aurora MySQL 3.05.2 (compatible with MySQL 8.0.32) - default for major version 8.0

 Parallel query is off by default. To enable it, use a DB instance parameter group with the `aurora_parallel_query` parameter enabled. [Learn more](#)

Templates

Choose a sample template to meet your use case.

☒ Production

Use defaults for high availability and fast, consistent performance.

☐ Dev/Test

This instance is intended for development use outside of a production environment.

In the following step remember the username and password. We require it when we access the DB Server in upcoming steps.

(In my case: username: admin, Password: shehan12345)

Settings

DB cluster identifier [Info](#)
Enter a name for your DB cluster. The name must be unique across all DB clusters owned by your AWS account in the current AWS Region.

The DB cluster identifier is case-insensitive, but is stored as all lowercase (as in "mydbcluster"). Constraints: 1 to 60 alphanumeric characters or hyphens. First character must be a letter. Can't contain two consecutive hyphens. Can't end with a hyphen.

▼ **Credentials Settings**

Master username [Info](#)
Type a login ID for the master user of your DB instance.

1 to 32 alphanumeric characters. The first character must be a letter.

Credentials management
You can use AWS Secrets Manager or manage your master user credentials.

☐ **Managed in AWS Secrets Manager - most secure**
RDS generates a password for you and manages it throughout its lifecycle using AWS Secrets Manager.

☒ **Self managed**
Create your own password or have RDS create a password that you manage.

☐ **Auto generate password**
Amazon RDS can generate a password for you, or you can specify your own password.

Master password [Info](#)

Password strength Neutral

Minimum constraints: At least 8 printable ASCII characters. Can't contain any of the following symbols: / ' " @

Confirm master password [Info](#)

Cluster storage configuration - new [Info](#)

Choose the storage configuration for the Aurora DB cluster that best fits your application's price predictability and price performance needs.

Configuration options
Database instance, storage, and I/O charges vary depending on the configuration. [Learn more](#)

☒ **Aurora Standard**

- Cost-effective pricing for many applications with moderate I/O usage (I/O costs <25% of total database costs).
- Pay-per-request I/O charges apply. DB instance and storage prices don't include I/O usage.

☐ **Aurora I/O-Optimized**

- Predictable pricing for all applications. Improved price performance for I/O-intensive applications (I/O costs >25% of total database costs).
- No additional charges for read/write I/O operations. DB instance and storage prices include I/O usage.

Instance configuration

The DB instance configuration options below are limited to those supported by the engine that you selected above.

DB instance class [Info](#)

▼ Hide filters

- ☒ Include previous generation classes
- ☐ Serverless v2
- ☐ Memory optimized classes (includes r classes)
- ☒ Burstable classes (includes t classes)

db.t3.medium
2 vCPUs 4 GiB RAM Network: 2,085 Mbps

Connectivity [Info](#)



Compute resource

Choose whether to set up a connection to a compute resource for this database. Setting up a connection will automatically change connectivity settings so that the compute resource can connect to this database.

- ☒ **Don't connect to an EC2 compute resource**
Don't set up a connection to a compute resource for this database. You can manually set up a connection to a compute resource later.
- ☐ **Connect to an EC2 compute resource**
Set up a connection to an EC2 compute resource for this database.

Network type [Info](#)

To use dual-stack mode, make sure that you associate an IPv6 CIDR block with a subnet in the VPC you specify.

- ☒ **IPv4**
Your resources can communicate only over the IPv4 addressing protocol.
- ☐ **Dual-stack mode**
Your resources can communicate over IPv4, IPv6, or both.

Virtual private cloud (VPC) [Info](#)

Choose the VPC. The VPC defines the virtual networking environment for this DB cluster.

Three-tier-project-VPC (vpc-0c733d1887af88883)
6 Subnets, 2 Availability Zones

Only VPCs with a corresponding DB subnet group are listed.

i After a database is created, you can't change its VPC.

DB subnet group [Info](#)

Choose the DB subnet group. The DB subnet group defines which subnets and IP ranges the DB cluster can use in the VPC that you selected.

db-subnet-group-three-tier-application
2 Subnets, 2 Availability Zones

Public access [Info](#)

- ☐ **Yes**
RDS assigns a public IP address to the cluster. Amazon EC2 instances and other resources outside of the VPC can connect to your cluster. Resources inside the VPC can also connect to the cluster. Choose one or more VPC security groups that specify which resources can connect to the cluster.
- ☒ **No**
RDS doesn't assign a public IP address to the cluster. Only Amazon EC2 instances and other resources inside the VPC can connect to your cluster. Choose one or more VPC security groups that specify which resources can connect to the cluster.

Existing VPC security groups

Choose one or more options ▼

DB-Tier-SG ✕

RDS Proxy
RDS Proxy is a fully managed, highly available database proxy that improves application scalability, resiliency, and security.

☐ Create an RDS Proxy [Info](#)
RDS automatically creates an IAM role and a Secrets Manager secret for the proxy. RDS Proxy has additional costs. For more information, see [Amazon RDS Proxy pricing](#).

Certificate authority - optional [Info](#)
Using a server certificate provides an extra layer of security by validating that the connection is being made to an Amazon database. It does so by checking the server certificate that is automatically installed on all databases that you provision.

rds-ca-rsa2048-g1 (default)
Expiry: May 20, 2061 ▼

If you don't select a certificate authority, RDS chooses one for you.

► **Additional configuration**

Monitoring

☐ **Enable Enhanced Monitoring**
Enabling Enhanced Monitoring metrics are useful when you want to see how different processes or threads use the CPU.

Keep other settings as default. This DB Server creation will cost you some money. But since we delete as soon as project is implemented, it will be few dollars.

Now Click **Create database**. It will take some time to create the server.

RDS > Databases

Databases (3) Group resources Refresh Modify Actions ▼ Restore from S3 Create database

Filter by databases

	DB identifier	Status	Role	Engine	Region & AZ	Size	Recommendations
☐	rds-three-tier	⌚ Creating	Regional cluster	Aurora MySQL	ap-south-1	2 instances	
☐	rds-three-tier-instance-1	⌚ Creating	Reader instance	Aurora MySQL	ap-south-1a	db.t3.medium	
☐	rds-three-tier-instance-1-ap-south-1b	⌚ Creating	Reader instance	Aurora MySQL	ap-south-1b	db.t3.medium	

Great we created the database server 😊👏

5. Create IAM Role

Our application should be Highly secured. In IAM Role we have to use two permission policies in upcoming steps. **S3 read only Access** for access the S3 bucket and **SSM managed instance core** for grants the necessary permissions for an EC2 instance to communicate with the Systems Manager service.

Let's create them

Search for IAM in the search bar. Go to **Roles > Create role**

The screenshot shows the 'Create role' wizard in the AWS IAM console. The 'Trusted entity type' section has five options: 'AWS service' (selected), 'AWS account', 'Web identity', 'SAML 2.0 federation', and 'Custom trust policy'. The 'Use case' section shows 'EC2' selected in the 'Service or use case' dropdown. Below, the 'EC2' use case is selected, with a description: 'Allows EC2 instances to call AWS services on your behalf.'

Click next. Then Search for **AmazonS3ReadOnlyAccess** and **AmazonSSMManagedInstanceCore** and select them. Click next.

The screenshot shows the 'Role details' and 'Permissions policy summary' sections of the 'Create role' wizard. The 'Role details' section has a 'Role name' field with 'IAM-role-three-tier-app' and a 'Description' field with 'Allows EC2 instances to call AWS services on your behalf.' The 'Permissions policy summary' section shows a table with two policies attached to the role.

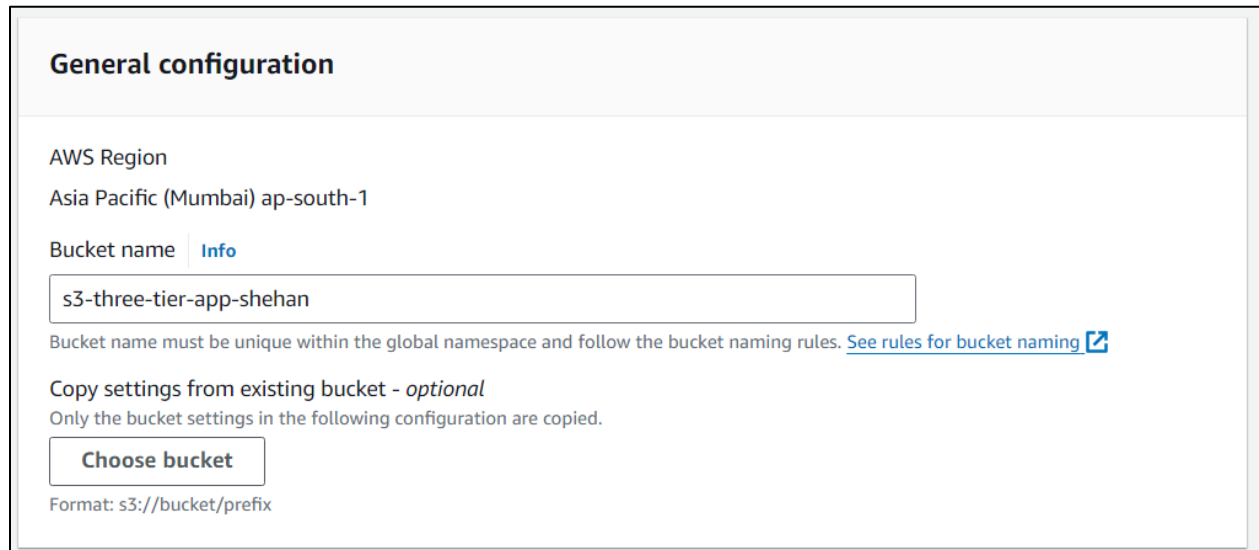
Policy name	Type	Attached as
AmazonS3ReadOnlyAccess	AWS managed	Permissions policy
AmazonSSMManagedInstanceCore	AWS managed	Permissions policy

Click Create role.

6. Create a S3 Bucket and upload the Source code

When we setup application on servers it is easy to if the source codes are in a s3 bucket

Go to search bar and search for S3. Then click **create bucket**. Bucket name should be unique one.



The screenshot shows the 'General configuration' section of the AWS S3 console. It includes the following fields and options:

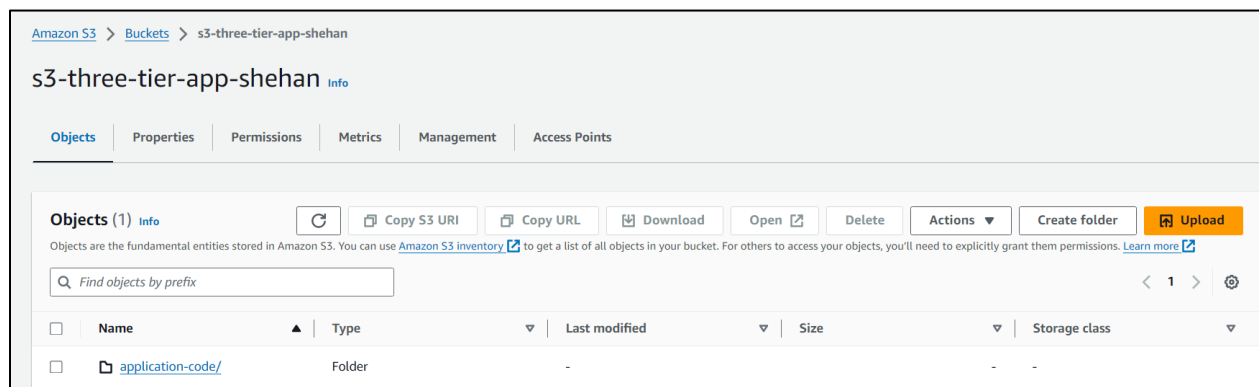
- AWS Region:** Asia Pacific (Mumbai) ap-south-1
- Bucket name:** s3-three-tier-app-shehan (with an 'Info' link next to it)
- Bucket name validation:** A message stating 'Bucket name must be unique within the global namespace and follow the bucket naming rules. [See rules for bucket naming](#)'
- Copy settings from existing bucket - optional:** A section with a note 'Only the bucket settings in the following configuration are copied.' and a 'Choose bucket' button.
- Format:** s3://bucket/prefix

Let all other options default and click create bucket.

Now go to the following repository and clone it in your pc. It contains all source code.

https://github.com/shehan404/Three_Tier_app_on_aws

Now go to the bucket and upload the application-code folder. There are some changes to make in the files. We will do it in the upcoming steps.



The screenshot shows the AWS S3 console interface for the bucket 's3-three-tier-app-shehan'. It includes the following elements:

- Navigation:** Amazon S3 > Buckets > s3-three-tier-app-shehan
- Bucket Name:** s3-three-tier-app-shehan (with an 'Info' link)
- Tabs:** Objects, Properties, Permissions, Metrics, Management, Access Points
- Objects (1):** A section with a search bar and a table of objects.
- Actions:** A row of buttons including 'Copy S3 URI', 'Copy URL', 'Download', 'Open', 'Delete', 'Actions', 'Create folder', and 'Upload'.
- Table:** A table with columns: Name, Type, Last modified, Size, and Storage class. It contains one entry: 'application-code/' (Folder).

Our application should be highly scalable. For achieve it we have to use Auto Scaling Groups. For automatically creating servers and it configurations we use Launch templates so we need Amazon Machine Images (AMI) of our servers. For that we are going to create a test server using EC2 instance and configure application in that and then create an AMI using it. In this way we implement our Web Servers and App Servers.

Let's do 🤖

7. Create Test App Server and setup it

Search for EC2 in the search bar. Go to **Instances > Launch instances**. Setup it as follows.

Name and tags [Info](#)

Name

Test-APP-Server

Add additional tags

▼ **Application and OS Images (Amazon Machine Image)** [Info](#)

An AMI is a template that contains the software configuration (operating system, application server, and applications) required to launch your instance. Search or Browse for AMIs if you don't see what you are looking for below

Q Search our full catalog including 1000s of application and OS images

Recents

Quick Start

Amazon Linux

aws

macOS

Mac

Ubuntu

ubuntu

Windows

Microsoft

Red Hat

Red Hat

SUSE Li

SUS

Browse more AMIs

Including AMIs from AWS, Marketplace and the Community

Amazon Machine Image (AMI)

Amazon Linux 2023 AMI

ami-0a4408457f9a03be3 (64-bit (x86), uefi-preferred) / ami-0741e326f07d8dfa8 (64-bit (Arm), uefi)

Free tier eligible

Virtualization: hvm

ENA enabled: true

Root device type: ebs

Description

Amazon Linux 2023 is a modern, general purpose Linux-based OS that comes with 5 years of long term support. It is optimized for AWS and designed to provide a secure, stable and high-performance execution environment to develop and run your cloud applications.

Architecture

64-bit (x86)

Boot mode

uefi-preferred

AMI ID

ami-0a4408457f9a03be3

Verified provider

Shehan Arampola | <https://www.linkedin.com/in/shehan-arampola> | <https://github.com/shehan404>

▼ Instance type [Info](#) | [Get advice](#)

Instance type

t2.micro

Family: t2 1 vCPU 1 GiB Memory Current generation: true
On-Demand Linux base pricing: 0.0124 USD per Hour
On-Demand Windows base pricing: 0.017 USD per Hour
On-Demand RHEL base pricing: 0.0268 USD per Hour
On-Demand SUSE base pricing: 0.0124 USD per Hour

Free tier eligible

☐ All generations

[Compare instance types](#)

Additional costs apply for AMIs with pre-installed software

▼ Key pair (login) [Info](#)

You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.

Key pair name - *required*

Proceed without a key pair (Not recommended)

Default value ▼

[Create new key pair](#)

▼ Network settings [Info](#)

VPC - *required* | [Info](#)

vpc-0c733d1887af88883 (Three-tier-project-VPC)

10.0.0.0/16

[Refresh](#)

Subnet | [Info](#)

subnet-0631d4ffa45df8f68

App-Tier-Subnet-1-Private

VPC: vpc-0c733d1887af88883 Owner: 654654498335
Availability Zone: ap-south-1a Zone type: Availability Zone
IP addresses available: 251 CIDR: 10.0.2.0/24

[Refresh](#) [Create new subnet](#)

Auto-assign public IP | [Info](#)

Disable

Firewall (security groups) | [Info](#)

A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

☐ Create security group

☒ Select existing security group

Common security groups [Info](#)

Select security groups

App-Tier-SG sg-06187cc79e5667ce9 ✕

VPC: vpc-0c733d1887af88883

[Refresh](#) [Compare security group rules](#)

Security groups that you add or remove here will be added to or removed from all your network interfaces.

[► Advanced network configuration](#)

▼ **Advanced details** Info

Domain join directory Info

Select ▼ [Create new directory](#)

IAM instance profile Info

IAM-role-three-tier-app
arn:aws:iam::654654498335:instance-profile/IAM-role-three-tier-app ▼ [Create new IAM profile](#)

Keep other settings as default and click Launch Instance.

Now we have to connect to the server and configure the application tier.

Before that let's do some changes in the source code. Go to the source code folder on your pc. Go to **application-code > app-tier**. Open **DbConfig.js** with your favorite editor. I use vs code.

```
application-code > app-tier > JS DbConfig.js > ...
1  module.exports = Object.freeze({
2    ...
3    DB_HOST: "",
4    DB_USER: "",
5    DB_PWD: "",
6    DB_DATABASE: "",
7  });
```

We have to change these values.

Let's see how to get the **DB_HOST**

Go to search and go to **RDS > Databases**. Select Writer instance. Under connectivity and security, you can find a value called Endpoint. Copy it and paste it as the value for DB_HOST.

RDS > Databases > rds-three-tier > rds-three-tier-instance-1

rds-three-tier-instance-1

Related

Filter by databases

DB identifier	Status	Role	Engine	Region & AZ	Size	Recommend
rds-three-tier	Available	Regional cluster	Aurora MySQL	ap-south-1	2 instances	
rds-three-tier-instance-1	Available	Writer instance	Aurora MySQL	ap-south-1a	db.t3.medium	
rds-three-tier-instance-1-ap-south-1b	Available	Reader instance	Aurora MySQL	ap-south-1b	db.t3.medium	

Connectivity & security | Monitoring | Logs & events | Configuration | Maintenance & backups | Tags | Recommendations

Connectivity & security

Endpoint & port	Networking	Security
Endpoint rds-three-tier-instance-1.c7gqk4m2uomf.ap-south-1.rds.amazonaws.com	Availability Zone ap-south-1a VPC Three-tier-project-VPC (vpc-0c733d1887af88883) Subnet group db-subnet-group-three-tier-application	VPC security groups DB-Tier-SG (sg-02b96a2ea03c8b652) Active Publicly accessible No Certificate authority Info
Port 3306		

DB_USER is the username for DB server (admin). DB_PWD is the password we assigned (shehan12345). DB_DATABASE is webappdb.

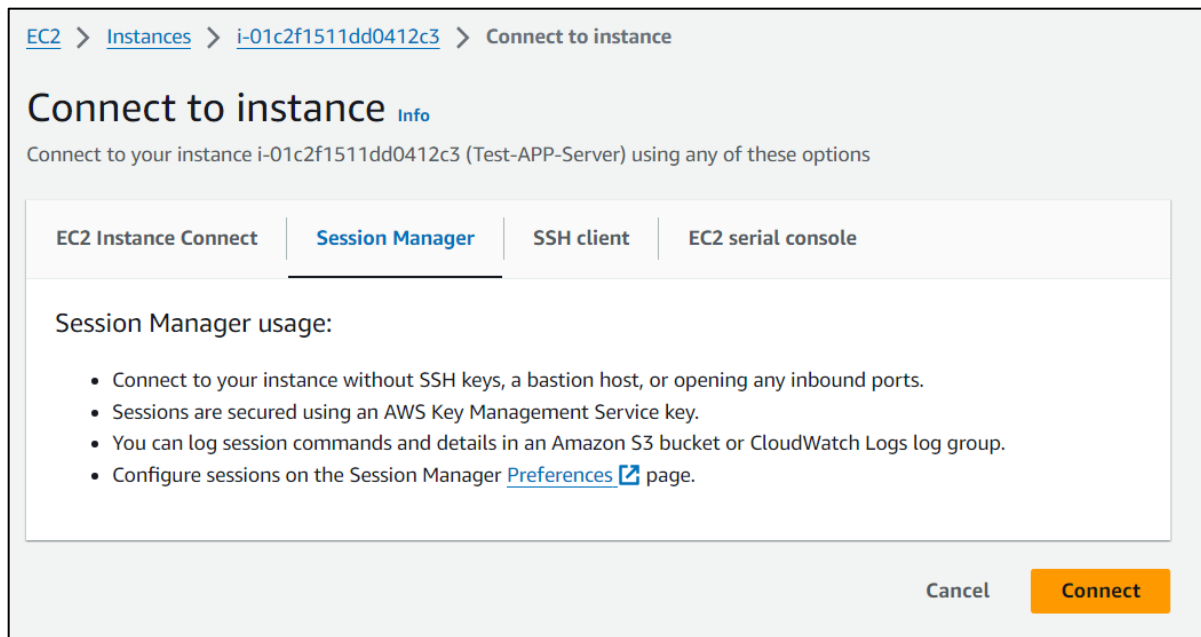
```
application-code > app-tier > DbConfig.js > ...
1  module.exports = Object.freeze({
2    DB_HOST: "rds-three-tier-instance-1.c7gqk4m2uomf.ap-south-1.rds.amazonaws.com",
3    DB_USER: "admin",
4    DB_PWD: "shehan12345",
5    DB_DATABASE: "webappdb",
6  });
7
```

Now go to the S3 Bucket we created. Go to **application-code > app-tier** in the s3 bucket. Delete the DbConfig.js and then upload our newly updated DbConfig.js file.

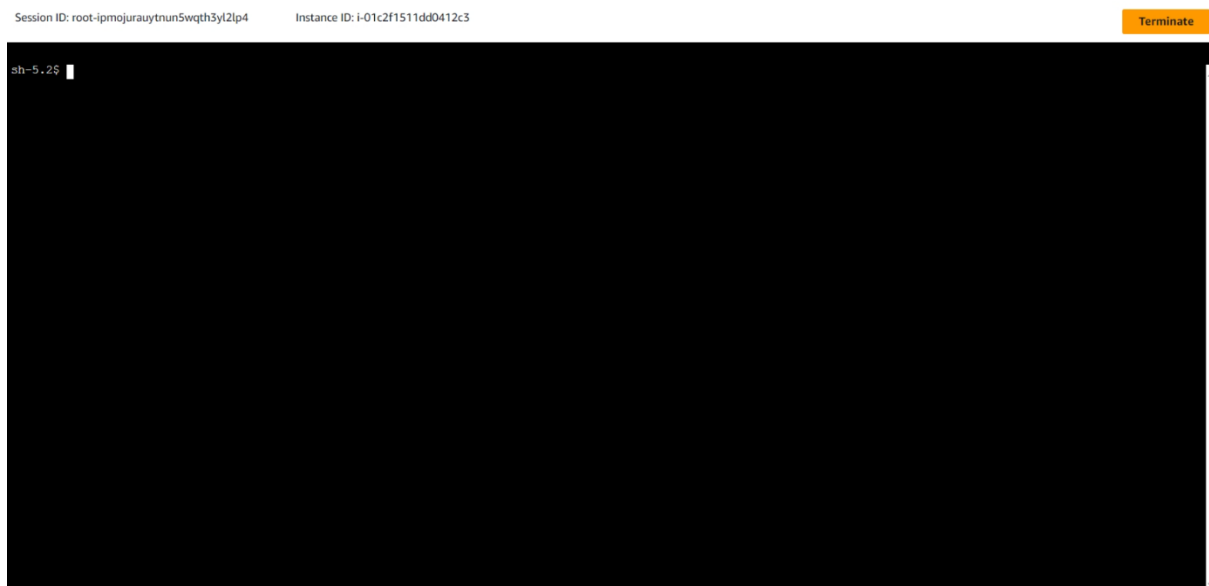
Cool 😊.

Now again go to EC2 instance and select Test-APP-Server we created click connect.

Go to Session Manager tab and click connect.



It will open a new tab like this. Which means you have connected to EC2 instance. IAM role we created let us connect to this EC2 instance even if it is in a private subnet.



Now let's configure the app-tier

First Switch user to the ec2-user by running `sudo -su ec2-user`

Now install the MySQL using following commands.

```
sudo wget https://dev.mysql.com/get/mysql80-community-release-el9-1.noarch.rpm
```

```
sudo dnf install mysql80-community-release-el9-1.noarch.rpm -y
```

```
sudo rpm --import https://repo.mysql.com/RPM-GPG-KEY-mysql-2023
```

```
sudo dnf install mysql-community-client -y
```

Run `mysql --version` to confirm it installed correctly

```
[ec2-user@ip-10-0-2-216 bin]$ mysql --version
mysql Ver 8.0.39 for Linux on x86_64 (MySQL Community Server - GPL)
[ec2-user@ip-10-0-2-216 bin]$
```

Now let's connect to Database server by running following command

```
mysql -h <RDS-Endpoint> -u <username> -p
```

RDS-Endpoint: Endpoint value of Writer instance copied from the RDS>Databases.

Username: admin

```
mysql -h rds-three-tier-instance-1.c7gqk4m2uomf.ap-south-1.rds.amazonaws.com -u admin -p
```

Hit enter and input the password (shehan12345). Enter. You will see the following output.

```
[ec2-user@ip-10-0-2-216 bin]$ mysql -h rds-three-tier-instance-1.c7gqk4m2uomf.ap-south-1.rds.amazonaws.com -u admin -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 804
Server version: 8.0.32 Source distribution

Copyright (c) 2000, 2024, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

Let's Create the database. Run following commands.

```
create DATABASE webappdb;
```

```
show databases;
```

```
use webappdb;
```

```
mysql> create DATABASE webappdb;
Query OK, 1 row affected (0.00 sec)

mysql> show databases;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| sys |
| webappdb |
+-----+
5 rows in set (0.00 sec)

mysql> use webappdb;
Database changed
mysql> 
```

Now Create a table by running following command

```
CREATE TABLE IF NOT EXISTS transactions(id INT NOT NULL
AUTO_INCREMENT, amount DECIMAL(10,2), description
VARCHAR(100), PRIMARY KEY(id));
```

```
mysql> CREATE TABLE IF NOT EXISTS transactions(id INT NOT NULL
-> AUTO_INCREMENT, amount DECIMAL(10,2), description
-> VARCHAR(100), PRIMARY KEY(id));
Query OK, 0 rows affected (0.02 sec)

mysql> show tables;
+-----+
| Tables_in_webappdb |
+-----+
| transactions |
+-----+
1 row in set (0.00 sec)

mysql> 
```

Let's create insert a sample value.

```
mysql> INSERT INTO transactions (amount,description) VALUES ('400','groceries');
Query OK, 1 row affected (0.00 sec)

mysql> SELECT * FROM transactions;
+----+-----+-----+
| id | amount | description |
+----+-----+-----+
| 1 | 400.00 | groceries |
+----+-----+-----+
1 row in set (0.00 sec)

mysql>
```

Type **exit** and hit Enter

Next we copy application tier codes from s3 bucket. Replace <S3_BUCKET_NAME> with your s3 bucket name.

```
cd /home/ec2-user
sudo aws s3 cp s3://<YOUR-S3-BUCKET-NAME>/application-code/app-tier app-tier
--recursive
```

```
cd app-tier
sudo chown -R ec2-user:ec2-user /home/ec2-user/app-tier
sudo chmod -R 755 /home/ec2-user/app-tier
```

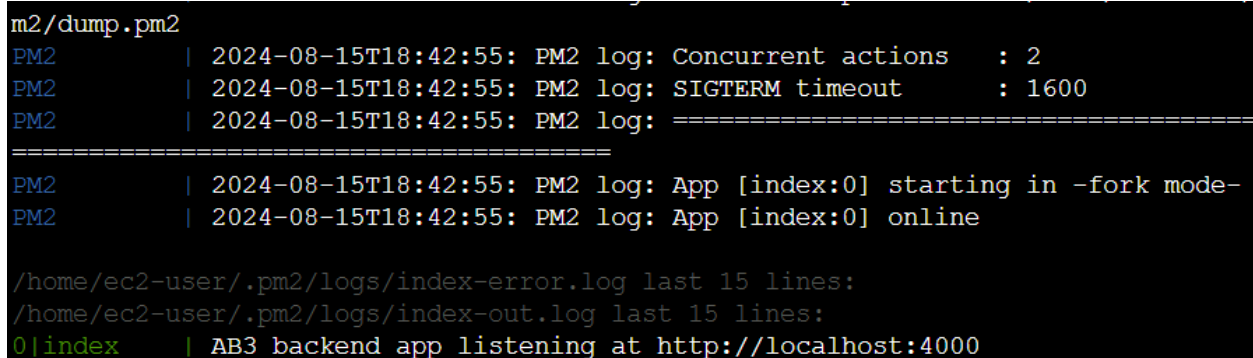
```
[ec2-user@ip-10-0-2-216 bin]$ cd /home/ec2-user
[ec2-user@ip-10-0-2-216 ~]$ sudo aws s3 cp s3://s3-three-tier-app-shehan/application-code/app-tier app-tier --recursive
download: s3://s3-three-tier-app-shehan/application-code/app-tier/DbConfig.js to app-tier/DbConfig.js
download: s3://s3-three-tier-app-shehan/application-code/app-tier/package.json to app-tier/package.json
download: s3://s3-three-tier-app-shehan/application-code/app-tier/README.md to app-tier/README.md
download: s3://s3-three-tier-app-shehan/application-code/app-tier/index.js to app-tier/index.js
download: s3://s3-three-tier-app-shehan/application-code/app-tier/TransactionService.js to app-tier/TransactionService.js
download: s3://s3-three-tier-app-shehan/application-code/app-tier/package-lock.json to app-tier/package-lock.json
[ec2-user@ip-10-0-2-216 ~]$ cd app-tier
[ec2-user@ip-10-0-2-216 app-tier]$ ls
DbConfig.js  TransactionService.js  package-lock.json
README.md    index.js              package.json
[ec2-user@ip-10-0-2-216 app-tier]$ sudo chown -R ec2-user:ec2-user /home/ec2-user/app-tier
[ec2-user@ip-10-0-2-216 app-tier]$ sudo chmod -R 755 /home/ec2-user/app-tier
[ec2-user@ip-10-0-2-216 app-tier]$
```

Now let's install Node.js

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh |  
bash  
  
source ~/.bashrc  
  
nvm install 16  
  
nvm use 16  
  
npm install -g pm2  
  
npm install  
  
npm audit fix
```

After that let's run the Index.js file and then save the configurations.

```
pm2 start index.js  
pm2 logs
```



A terminal window with a dark background showing the output of 'pm2 logs'. The logs include PM2 status messages, a separator line, and application logs indicating the app is starting in -fork mode and is now online. At the bottom, it shows the last 15 lines of the error and output logs, with the output log showing 'AB3 backend app listening at http://localhost:4000'.

```
m2/dump.pm2  
PM2      | 2024-08-15T18:42:55: PM2 log: Concurrent actions    : 2  
PM2      | 2024-08-15T18:42:55: PM2 log: SIGTERM timeout      : 1600  
PM2      | 2024-08-15T18:42:55: PM2 log: =====  
=====  
PM2      | 2024-08-15T18:42:55: PM2 log: App [index:0] starting in -fork mode-  
PM2      | 2024-08-15T18:42:55: PM2 log: App [index:0] online  
  
/home/ec2-user/.pm2/logs/index-error.log last 15 lines:  
/home/ec2-user/.pm2/logs/index-out.log last 15 lines:  
0|index   | AB3 backend app listening at http://localhost:4000
```

If you got output like this. The backend is connected to port 4000. Press Ctrl+c to exit the loop. Then run the following commands.

```
pm2 startup  
  
sudo env PATH=$PATH:/home/ec2-user/.nvm/versions/node/v16.20.2/bin /home/ec2-  
user/.nvm/versions/node/v16.20.2/lib/node_modules/pm2/bin/pm2 startup systemd  
-u ec2-user --hp /home/ec2-user  
  
pm2 save  
  
curl http://localhost:4000/health
```

```
[PM2] Remove init script via:
$ pm2 unstartup systemd
[ec2-user@ip-10-0-2-216 app-tier]$ pm2 save
urrent configuration)
[PM2] Saving current process list...
[PM2] Successfully saved in /home/ec2-user/.pm2/dump.pm2
[ec2-user@ip-10-0-2-216 app-tier]$ curl http://localhost:4000/health check)
"This is the health check"[ec2-user@ip-10-0-2-216 app-tier]$
```

Cool 😊 You successfully setup the app server

8. Create AMI of App-Server

Next Let's create an image from this Test-APP-Server instance.

Go to instances choose Test-APP-Server go to **Actions > Image and templates > Create image**.

Instance ID
i-01c2f1511dd0412c3 (Test-APP-Server)

Image name
APP-server-image
Maximum 127 characters. Can't be modified after creation.

Image description - optional
APP-server-image
Maximum 255 characters

☒ Reboot instance
When selected, Amazon EC2 reboots the instance so that data is at rest when snapshots of the attached volumes are taken. This ensures data consistency.

Instance volumes

Storage type	Device	Snapshot	Size	Volume type	IOPS	Throughput	Delete on termination	Encrypted
EBS	/...	Create new snapshot	8	EBS General Purpose	3000		☒ Enable	☐ Enable

Add volume

During the image creation process, Amazon EC2 creates a snapshot of each of the above volumes.

Tags - optional
A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

☒ Tag image and snapshots together
Tag the image and the snapshots with the same tag.
☐ Tag image and snapshots separately
Tag the image and the snapshots with different tags.

Key Value - optional

X
 X

If you go to AMIs section, you can see that App-Server AMI is created

9. Create Target Group, Launch Template, Internal Load Balancer for App Tier

Create Target Group

Go to EC2 > Target Groups > Create Target Group change only following settings.
Click Next. Click Create target group.

Target group name

A maximum of 32 alphanumeric characters including hyphens are allowed, but the name must not begin or end with a hyphen.

Protocol : Port

Choose a protocol for your target group that corresponds to the Load Balancer type that will route traffic to it. Some protocols now include anomaly detection for the targets and you can set mitigation options once your target group is created. This choice cannot be changed after creation

HTTP

4000

1-65535

IP address type

Only targets with the indicated IP address type can be registered to this target group.

☒ **IPv4**

Each instance has a default network interface (eth0) that is assigned the primary private IPv4 address. The instance's primary private IPv4 address is the one that will be applied to the target.

☐ **IPv6**

Each instance you register must have an assigned primary IPv6 address. This is configured on the instance's default network interface (eth0). [Learn more](#)

VPC

Select the VPC with the instances that you want to include in the target group. Only VPCs that support the IP address type selected above are available in this list.

Three-tier-project-VPC

vpc-0c733d1887af88883

IPv4 VPC CIDR: 10.0.0.0/16

Health check path

Use the default path of "/" to perform health checks on the root, or specify a custom path if preferred.

Up to 1024 characters allowed.

Advanced health check settings

[Restore defaults](#)

Health check port

The port the load balancer uses when performing health checks on targets. By default, the health check port is the same as the target group's traffic port. However, you can specify a different port as an override.

☒ **Traffic port**

☐ **Override**

Healthy threshold

The number of consecutive health check successes required before considering an unhealthy target healthy.

2

2-10

Unhealthy threshold

The number of consecutive health check failures required before considering a target unhealthy.

2

2-10

Timeout

The amount of time, in seconds, during which no response means a failed health check.

2

seconds

2-120

Interval

The approximate amount of time between health checks of an individual target

5

seconds

5-300

Success codes

The HTTP codes to use when checking for a successful response from a target. You can specify multiple values (for example, "200,202") or a range of values (for example, "200-299").

Create Launch Template

Go to EC2 > Launch Templates > Create launch template.

Launch template name and description

Launch template name - required

Must be unique to this account. Max 128 chars. No spaces or special characters like '&', '*', '@'.

Template version description

Max 255 chars

Auto Scaling guidance | [Info](#)

Select this if you intend to use this template with EC2 Auto Scaling

☒ **Provide guidance to help me set up a template that I can use with EC2 Auto Scaling**

▼ Application and OS Images (Amazon Machine Image) - required [Info](#)

An AMI is a template that contains the software configuration (operating system, application server, and applications) required to launch your instance. Search or Browse for AMIs if you don't see what you are looking for below

Recents

My AMIs

Quick Start

☒ Owned by me

☐ Shared with me

Browse more AMIs

Including AMIs from AWS, Marketplace and the Community

Amazon Machine Image (AMI)

APP-server-image

ami-0c0e49a4ee72ae2c5

2024-08-15T19:36:08.000Z

Virtualization: hvm

ENA enabled: true

Root device type: ebs

▼ Instance type [Info](#) | [Get advice](#)

Advanced

Instance type

t2.micro

Family: t2 1 vCPU 1 GiB Memory Current generation: true

On-Demand Linux base pricing: 0.0124 USD per Hour

On-Demand Windows base pricing: 0.017 USD per Hour

On-Demand RHEL base pricing: 0.0268 USD per Hour

On-Demand SUSE base pricing: 0.0124 USD per Hour

Free tier eligible

☐ All generations

[Compare instance types](#)

Additional costs apply for AMIs with pre-installed software

▼ Key pair (login) [Info](#)

You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.

Key pair name

Don't include in launch template

Create new key pair

▼ Network settings [Info](#)

Subnet [Info](#)

Don't include in launch template

[Create new subnet](#)

When you specify a subnet, a network interface is automatically added to your template.

Firewall (security groups) [Info](#)

A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

☒ Select existing security group

☐ Create security group

Security groups [Info](#)

Select security groups

App-Tier-SG sg-06187cc79e5667ce9 ✕

VPC: vpc-0c733d1887af88883

[Compare security group rules](#)

[► Advanced network configuration](#)

Shehan Arampola | <https://www.linkedin.com/in/shehan-arampola> | <https://github.com/shehan404>

▼
Advanced details
Info

IAM instance profile
Info

IAM-role-three-tier-app
▼

arn:aws:iam::654654498335:instance-profile/IAM-role-three-tier-app

Create new IAM profile

Hostname type
Info

Don't include in launch template
▼

Create Application Load Balancer (Internal)

Go to EC2 > Load Balancer > Create Load Balancer create application load balancer.

Basic configuration

Load balancer name

Name must be unique within your AWS account and can't be changed after the load balancer is created.

Application-load-balancer

A maximum of 32 alphanumeric characters including hyphens are allowed, but the name must not begin or end with a hyphen.

Scheme
Info

Scheme can't be changed after the load balancer is created.

☐ Internet-facing

An internet-facing load balancer routes requests from clients over the internet to targets. Requires a public subnet. [Learn more](#)

☒ Internal

An internal load balancer routes requests from clients to targets using private IP addresses. Compatible with the IPv4 and Dualstack IP address types.

Load balancer IP address type
Info

Select the front-end IP address type to assign to the load balancer. The VPC and subnets mapped to this load balancer must include the selected IP address types. Public IPv4 addresses have an additional cost.

☒ IPv4

Includes only IPv4 addresses.

☐ Dualstack

Includes IPv4 and IPv6 addresses.

Network mapping
Info

The load balancer routes traffic to targets in the selected subnets, and in accordance with your IP address settings.

VPC
Info

The load balancer will exist and scale within the selected VPC. The selected VPC is also where the load balancer targets must be hosted unless routing to Lambda or on-premises targets, or if using VPC peering. To confirm the VPC for your targets, view [target groups](#). For a new VPC, [create a VPC](#).

Three-tier-project-VPC
▼

vpc-0c733d1887af88883

IPv4 VPC CIDR: 10.0.0.0/16

Mappings
Info

Select at least two Availability Zones and one subnet per zone. The load balancer routes traffic to targets in these Availability Zones only. Availability Zones that are not supported by the load balancer or the VPC are not available for selection.

Availability Zones

☒ ap-south-1a (aps1-az1)

Subnet

subnet-0631d4ffa45df8f68
▼

App-Tier-Subnet-1-Private

IPv4 subnet CIDR: 10.0.2.0/24

IPv4 address

Assigned from CIDR 10.0.2.0/24

☒ ap-south-1b (aps1-az3)

Subnet

subnet-0925e6911290fbad0
▼

App-Tier-Subnet-2-Private

IPv4 subnet CIDR: 10.0.5.0/24

IPv4 address

Assigned from CIDR 10.0.5.0/24

Security groups [Info](#)

A security group is a set of firewall rules that control the traffic to your load balancer. Select an existing security group, or you can [create a new security group](#).

Security groups

Select up to 5 security groups

Internal-Load-Balancer-SG ✕

sg-0065f4ac04226f8c8 VPC: vpc-0c733d1887af88883

Listeners and routing [Info](#)

A listener is a process that checks for connection requests using the port and protocol you configure. The rules that you define for a listener determine how the load balancer routes requests to its registered targets.

▼ Listener HTTP:80 Remove

Protocol Port Default action [Info](#)

HTTP : 80 Forward to App-Server-TG ✕

1-65535 [Create target group](#)

Listener tags - optional

Consider adding tags to your listener. Tags enable you to categorize your AWS resources so you can more easily manage them.

Add listener tag

You can add up to 50 more tags.

Add listener

10. Create Test Web Server and setup it

We have to follow the same process as followed in App server. Only EC2 configuration part is different.

First go to the source code folder in your PC. Open **nginx.conf** file in an editor

Now go to the internal load balancer we created and copy the **DNS name** of it.

Go to the nginx.conf file and past it in the following section.

```
#proxy for internal lb
location /api/{
    proxy_pass http://<Internal load balancer DNS name>:80;
}
```

```
#proxy for internal lb
location /api/{
    proxy_pass http://internal-Application-load-balancer-1096585234.ap-south-1.elb.amazonaws.com:80;
}
```

Now go to S3 bucket and delete the **nginx.conf** file and upload the new one.

Now got EC2 instance > Launch instance and create a instance for Test Web Server. Change name (Test-Web-Server), VPC, Subnet(Web-Tier-Subnet-1-Pbulic), Security group (Web-Tier-SG) and also select IAM role under advanced details. Launch instance.

Now Connect to the instance and do configurations as follows.

Switch users

```
sudo -su ec2-user  
cd /home/ec2-user
```

Copy content from the S3 bucket

```
sudo aws s3 cp s3://<S3_BUCKET_NAME>/application-code/web-tier web-tier --recursive  
sudo chown -R ec2-user:ec2-user /home/ec2-user  
sudo chmod -R 755 /home/ec2-user
```

Install Node js

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh |  
bash  
source ~/.bashrc  
nvm install 16  
nvm use 16  
cd /home/ec2-user/web-tier  
npm install
```

Build the app

```
cd /home/ec2-user/web-tier  
npm run build
```

Install nginx (Webserver)

```
sudo yum install nginx -y
cd /etc/nginx
sudo mv nginx.conf nginx-backup.conf

sudo aws s3 cp s3://<S3_BUCKET_NAME>/application-code/nginx.conf .
sudo chmod -R 755 /home/ec2-user
sudo service nginx restart
sudo chkconfig nginx on
```

11. Create AMI of Web-Server

Go to instances and select Test-Web-Server > Actions > Image and Template > Create image.

12. Create Target Group, Launch Template, External Load Balancer for App Tier

Same process

Create Target Group

Change Protocol: Port to 80. Select VPC. Give name. Create.

Create Launch Template

Choose Web-Server-Image for AMI and choose Web-Tier-SG. Other settings are same as above app tier.

Create External Load Balancer

Choose Scheme as Internet-facing. Choose vpc. Choose Availability Zones of web-tier. Choose External load balancer Security Group. Choose target group in Listeners and routing. Create load balancer.

13. Create Auto Scaling groups

Auto Scaling group for App Tier

Go to EC2 > Create Auto Scaling group

Name

Auto Scaling group name

Enter a name to identify the group.

App-Tier-ASG

Must be unique to this account in the current Region and no more than 255 characters.

Launch template [Info](#)

For accounts created after May 31, 2023, the EC2 console only supports creating Auto Scaling groups with launch templates. Creating Auto Scaling groups with launch configurations is not recommended but still available via the CLI and API until December 31, 2023.

Launch template

Choose a launch template that contains the instance-level settings, such as the Amazon Machine Image (AMI), instance type, key pair, and security groups.

App-Server-Launch-template

Create a launch template [↗](#)

Version

Network [Info](#)

For most applications, you can use multiple Availability Zones and let EC2 Auto Scaling balance your instances across the zones. The default VPC and default subnets are suitable for getting started quickly.

VPC

Choose the VPC that defines the virtual network for your Auto Scaling group.

vpc-0c733d1887af88883 (Three-tier-project-VPC)

10.0.0.0/16

Create a VPC [↗](#)

Availability Zones and subnets

Define which Availability Zones and subnets your Auto Scaling group can use in the chosen VPC.

Select Availability Zones and subnets

ap-south-1a | subnet-0631d4ffa45df8f68 (App-Tier-Subnet-1-Private)

10.0.2.0/24

ap-south-1b | subnet-0925e6911290fbad0 (App-Tier-Subnet-2-Private)

10.0.5.0/24

Create a subnet [↗](#)

Shehan Arampola | <https://www.linkedin.com/in/shehan-arampola> | <https://github.com/shehan404>

Load balancing [Info](#)

Use the options below to attach your Auto Scaling group to an existing load balancer, or to a new load balancer that you define.

☐ No load balancer
Traffic to your Auto Scaling group will not be fronted by a load balancer.

☒ Attach to an existing load balancer
Choose from your existing load balancers.

☐ Attach to a new load balancer
Quickly create a basic load balancer to attach to your Auto Scaling group.

Attach to an existing load balancer
 Select the load balancers that you want to attach to your Auto Scaling group.

☒ Choose from your load balancer target groups
 This option allows you to attach Application, Network, or Gateway Load Balancers.

☐ Choose from Classic Load Balancers

Existing load balancer target groups
Only instance target groups that belong to the same VPC as your Auto Scaling group are available for selection.

Select target groups

App-Server-TG | HTTP

Application Load Balancer: Application-load-balancer

×

Group size [Info](#)

Set the initial size of the Auto Scaling group. After creating the group, you can change its size to meet demand, either manually or by using automatic scaling.

Desired capacity type
 Choose the unit of measurement for the desired capacity value. vCPUs and Memory(GiB) are only supported for mixed instances groups configured with a set of instance attributes.

Units (number of instances)

▼

Desired capacity
 Specify your group size.

2

Scaling [Info](#)

You can resize your Auto Scaling group manually or automatically to meet changes in demand.

Scaling limits
 Set limits on how much your desired capacity can be increased or decreased.

Min desired capacity

2

Equal or less than desired capacity

Max desired capacity

2

Equal or greater than desired capacity

Tags (1)

Key

Value - optional

Tag new instances

Name

App-Server

☒

Remove

Add tag

49 remaining

Cancel

Previous

Next

Keep other services as default and create Auto scaling group.

Shehan Arampola | <https://www.linkedin.com/in/shehan-arampola> | <https://github.com/shehan404>

Now if you check the instance list you Can see that two App Servers are creating

Instances (4) Info									
Find Instance by attribute or tag (case-sensitive) All states < 1 > ⚙️									
☐	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS	
☐	App-Sever	i-01246dbb28787bdf4	Pending	t2.micro	-	View alarms +	ap-south-1b	-	-
☐	App-Sever	i-0010aaf8ad8aad9c	Pending	t2.micro	-	View alarms +	ap-south-1a	-	-
☐	Test-APP-Server	i-01c2f1511dd0412c3	Running	t2.micro	2/2 checks passec	View alarms +	ap-south-1a	-	-
☐	Test-Web-Server	i-05482d8322e1d65ee	Running	t2.micro	2/2 checks passec	View alarms +	ap-south-1a	-	4

Auto Scaling group for Web Tier

The process is the same. Choose components related to Web Tier.

After you are creating it you can see 4 instance and test instance on the list. Now you can delete the test instances.

Instances (6) Info									
Find Instance by attribute or tag (case-sensitive) All states < 1 > ⚙️									
☐	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS	
☐	Web-Server	i-0199c73268f90b781	Pending	t2.micro	-	View alarms +	ap-south-1a	-	6
☐	Web-Server	i-0b5d5adc21fecdef4	Pending	t2.micro	-	View alarms +	ap-south-1b	-	1
☐	Test-Web-Server	i-05482d8322e1d65ee	Running	t2.micro	2/2 checks passec	View alarms +	ap-south-1a	-	4
☐	Test-APP-Server	i-01c2f1511dd0412c3	Running	t2.micro	2/2 checks passec	View alarms +	ap-south-1a	-	-
☐	App-Sever	i-0010aaf8ad8aad9c	Running	t2.micro	2/2 checks passec	View alarms +	ap-south-1a	-	-
☐	App-Sever	i-01246dbb28787bdf4	Running	t2.micro	2/2 checks passec	View alarms +	ap-south-1b	-	-

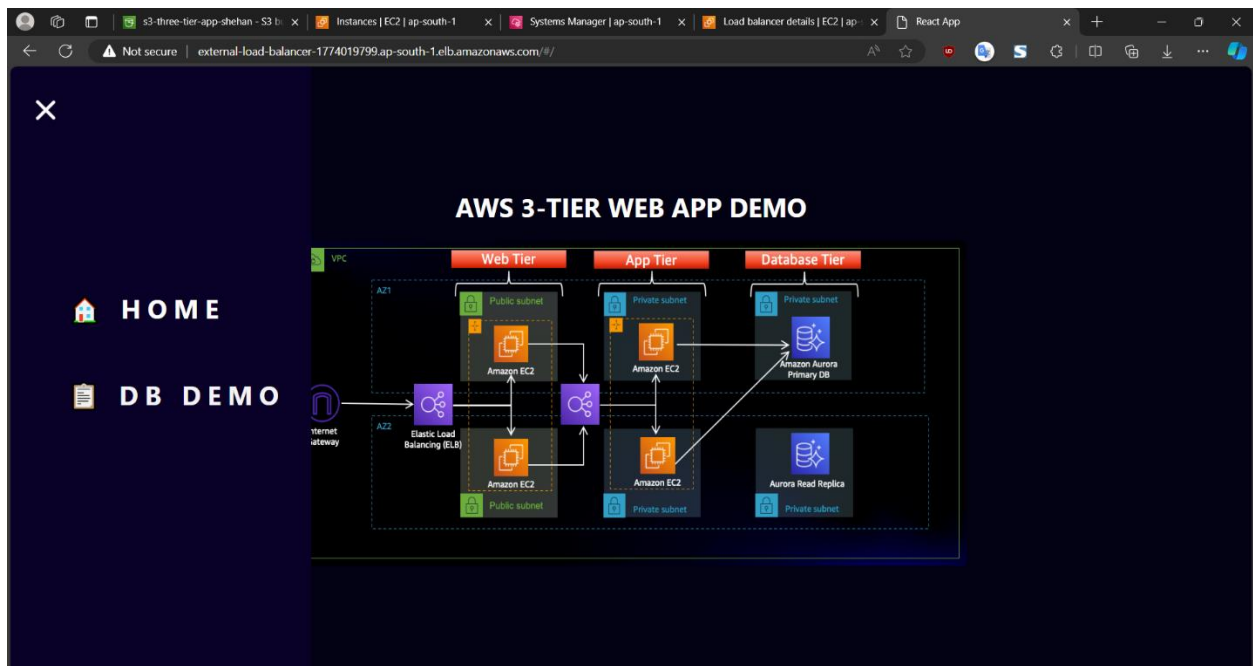
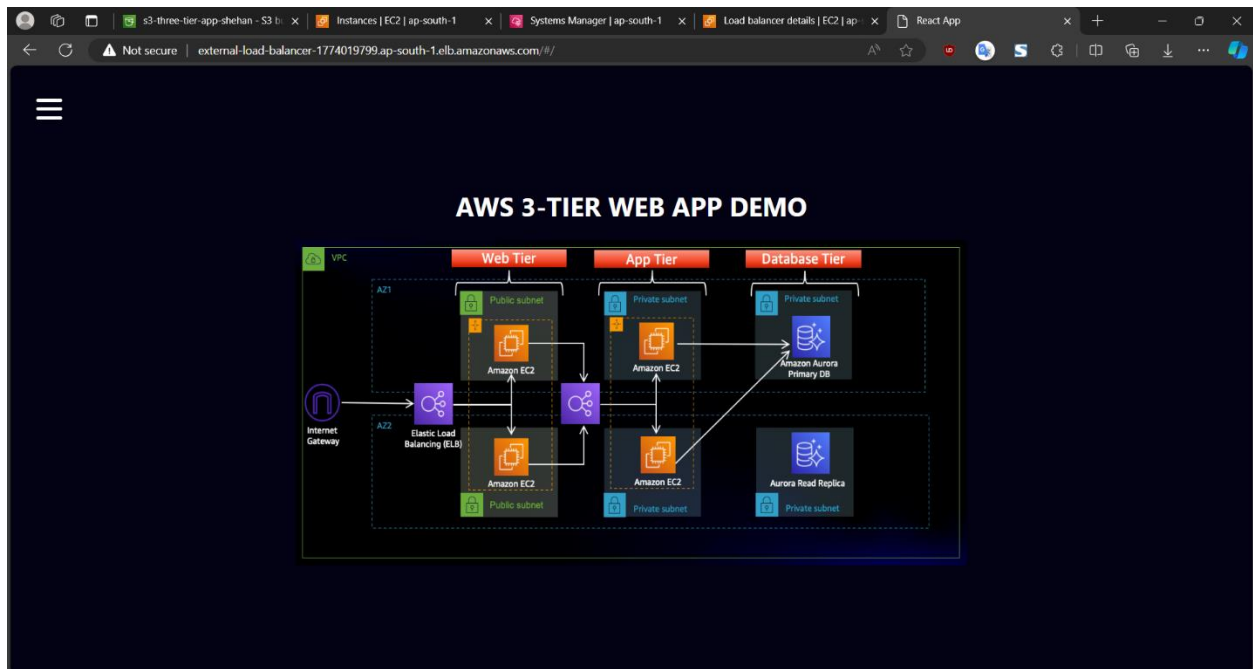
Cool 😊 You did a great job 🙌

Now We Can access our Web application.

14. Access the Application

Go to **EC2 > Load Balancers**. Select External Load Balancer. Copy the DNS name of it.

Paste It into your Browser. You will see the Application is running 🤖



Go to DB DEMO section and check whether DB is working.

AURORA DATABASE DEMO PAGE

ID	AMOUNT	DESC
ADD		
1	400	groceries

Here you can see the Example data we entered when setup the DB which means it receiving data from DB.

AURORA DATABASE DEMO PAGE

DEL

ID	AMOUNT	DESC
ADD		
4	100	groceries
5	50	ice cream
6	150	Food

You can add and delete items 😊

Congratulations!!! You have successfully implemented Three Tier Web Application on AWS 🚀👏😊

Note: In hear we use External Load Balancer DNS name as the URL. But it is not user friendly. So, We can purchase a domain name from domain name service and setup it using Route 53 to access the web site using that domain name.

15. Cleaning the Infrastructure

Since some services we used charge cost we have to delete our demo project.

Let's **delete** them according to the following order.

1. Auto Scaling Groups
2. In Application Load Balancers delete listeners first and then Load balancers
3. Target Groups
4. Launch Templates
5. AMIs
6. Snapshots
7. EC2 Instances
8. Aurora Database (First go to modify and disable delete protection. Then when deleting do not create snapshot. Tick I acknowledge)
9. NAT gateways
10. Elastic Ips
11. Route Tables (Remove all subnet associations first)
12. Internet Gateways
13. VPC
14. S3 Bucket
15. IAM Role